

1 LoRaWAN® End-Device and Network Server Interoperability Test Plan**2 NOTICE OF USE AND DISCLOSURE**

3 Copyright © LoRa Alliance, Inc. (2025). All rights reserved.

4 The information within this document is the property of the LoRa Alliance ("The Alliance") and its use and disclosure
5 are subject to LoRa Alliance Corporate Bylaws, Intellectual Property Rights (IPR) Policy and Membership Agreements.

6 Elements of LoRa Alliance specifications may be subject to third-party intellectual property rights, including without
7 limitation, patent, copyright or trademark rights (such a third party may or may not be a member of the LoRa
8 Alliance). The Alliance is not responsible and shall not be held responsible in any manner for identifying or failing to
9 identify any or all such third-party intellectual property rights.

10 This document and the information contained herein are provided on an "AS IS" basis and THE ALLIANCE DISCLAIMS
11 ALL WARRANTIES EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO (A) ANY WARRANTY THAT
12 THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY RIGHTS OF THIRD PARTIES (INCLUDING
13 WITHOUT LIMITATION ANY INTELLECTUAL PROPERTY RIGHTS INCLUDING PATENT, COPYRIGHT OR
14 TRADEMARK RIGHTS) OR (B) ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A
15 PARTICULAR PURPOSE, TITLE OR NONINFRINGEMENT.

16 IN NO EVENT WILL THE ALLIANCE BE LIABLE FOR ANY LOSS OF PROFITS, LOSS OF BUSINESS, LOSS
17 OF USE OF DATA, INTERRUPTION OF BUSINESS, OR FOR ANY OTHER DIRECT, INDIRECT, SPECIAL OR
18 EXEMPLARY, INCIDENTAL, PUNITIVE OR CONSEQUENTIAL DAMAGES OF ANY KIND, IN CONTRACT OR
19 IN TORT, IN CONNECTION WITH THIS DOCUMENT OR THE INFORMATION CONTAINED HEREIN, EVEN IF
20 ADVISED OF THE POSSIBILITY OF SUCH LOSS OR DAMAGE.

21 The above notice and this paragraph must be included on all copies of this document.

22 LoRa Alliance ®
23 5177 Brandin Court
24 Fremont, CA 94538
25 United States

26 *Note: LoRa Alliance® and LoRaWAN® are trademarks of the LoRa Alliance, used by permission. All company, brand*
27 *and product names may be trademarks that are the sole property of their respective owners.*



28

29 **LoRaWAN® End-Device and Network Server Interoperability Test Plan**

30 Authored by the LoRa Alliance Certification Committee Interoperability Workgroup

31 **Working Chair:**

32 R. STILLINGER (ZENNER)

33 **Editors:**

34 N. BHASKARAN (SpireSpark), I. CHARALAMPIDIS (KUDZU)

35 **Contributors:**

36 N. BHASKARAN (SpireSpark), I. CHARALAMPIDIS (KUDZU), P. HAMPTON (LoRa Alliance), M. LUIS (Semtech),

37 R. STILLINGER (ZENNER), D. THOLL (Tektelic), A. Yegin (LoRa Alliance)

38 Version: 1.0.0

39 Date: APR. 2025

40 Status: FINAL

41 Contents

42	LoRaWAN® End-Device and Network Server Interoperability Test Plan	2
43	Revision History	4
44	Glossary	4
45	Abbreviations & Acronyms	4
46	Definitions of terms used in this document	4
47	1 Introduction	5
48	1.1 Scope	5
49	1.2 Purpose of Testing	5
50	2 Test Setup	5
51	2.1 Test Environment Components	5
52	2.1.1 LoRaWAN Network Server	5
53	2.1.2 LoRaWAN End-Devices	6
54	2.1.3 LoRaWAN Gateway	6
55	2.1.4 Testing Tools and Software	6
56	2.2 Infrastructure Setup	6
57	2.2.1 Placement of Elements	6
58	2.2.2 Device Configuration	6
59	2.2.3 Frequency Plan	6
60	2.3 Implicitly Tested Test Cases	7
61	3 Interoperability Test Cases	7
62	3.1 Test Setup - Single End-Device - LNS Test Cases	7
63	3.1.1 Components	7
64	3.1.2 Capabilities Required	8
65	3.1.3 Events Expected	8
66	3.1.4 Prerequisites	8
67	3.2 Test Cases	9
68	3.2.1 Activation by Personalisation test	9
69	3.2.2 Over The Air Activation- Join-Rejoin Process	9
70	3.2.3 Uplink Transmission	10
71	3.2.4 Downlink Transmission	11
72	3.2.5 Class B tests	12
73	3.2.6 Class C tests	12
74	4 Appendix	13
75	4.1 Capability Tree	13
76	4.1.1 CAP.1 : End device capabilities	13
77	4.1.2 CAP.2 : Device-Level LNS Capabilities	13
78	4.1.3 CAP.3 : Application Server capabilities	14
79	4.2 Events Tree	14
80	4.2.1 EVT.1 : End device event	14
81	4.2.2 EVT.2 : LNS Events	16
82	4.3 Shared Event Fields	17
83	4.4 Full session context elements	18

84 Revision History

Version	Author	Updates	Date
Drafts	Paul Hampton	Initial draft	2024-09-17
Drafts	Ioannis Charalampidis	Initial markdown version	2024-11-20
0.10.0	Nisha Bhaskaran	Moved all tests from word doc to markdown	2024-12-10
0.10.1	Nisha Bhaskaran	Added tests based on ThinkPark Interop Test Plan	2024-01-15
0.10.2	Nisha Bhaskaran	Removed tests which require changes to the NS, ED to execute the tests. Incorporated comments from TC review	2025-03-20
1.0.0	Nisha Bhaskaran	First release	2025-04-03

85 Glossary

- **Certification Logo:** LoRa Alliance defined logo that can be displayed on the Certified product and any documentation and marketing information about the End-Device.
- **LoRaWAN Test House:** Organization and corresponding facility accredited by the LoRa Alliance to perform Certification testing.
- **End Device:** Devices submitted to a LoRaWAN Test House for Certification.

91 Abbreviations & Acronyms

ABP	Activation by Personalization
ADR	Adaptive Data Rate
AS	Application Server
ATL	Authorized Test Lab
DR	Data Rate
DUT	Device Under Test
GW	Gateway
JS	Join Server
LCTT	LoRaWAN Conformance Test Tool
MAC	Media Access Control
NS	Network Server
OTAA	Over-the-Air Activation

92 Definitions of terms used in this document

XXXX	X is a valid number which would vary based on the region being tested
DC	Dynamic Channel
FC	Fixed Channel
MinDR	Minimum Data Rate for the region tested, as specified in the Regional Parameters Specification [2]
Max125kHzDR	Maximum Data Rate using 125 kHz bandwidth for the region tested, as specified in the Regional Parameters Specification [2]
Nb	Number
R	Repeat until the condition specified within brackets [] is completed
LoRaWAN Testbed	A controlled environment or infrastructure specifically designed to evaluate, test, and validate the interoperability of LoRaWAN devices and networks.

93 1 Introduction

94 This version of the LoRaWAN® End-Device and Network Server Interoperability Test Plan is intended to verify
 95 the interoperability of End-Devices, Network Servers and Gateways against each other. The tests contained in this
 96 document are designed to be performed without any modification to the Network Server or End-Device. The testing
 97 is not intended to be conducted as part of Certification testing. A human operator may be required to run the tests
 98 and it is not intended to be automated.

99 1.1 Scope

100 This test plan describes specific test cases which can be executed without changing conditions which would cause any
 101 of the network elements to act in a way which is not normal in everyday operation.

102 1.2 Purpose of Testing

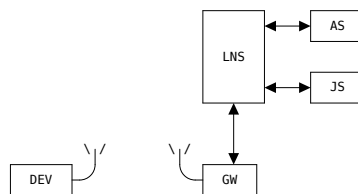
- 103 ■ **Ensuring Compatibility:** LoRaWAN is a standardized protocol, but different manufacturers may implement
 104 features in varying ways. Interoperability testing ensures that devices from different manufacturers can
 105 communicate effectively and operate predictably within the same network. Interoperability is not a substitute
 106 for compliance testing.
- 107 ■ **Validation of Functionality:** With interoperability testing, the functionality of the End-Device can be verified
 108 in the context of a given Network Server and its infrastructure. This includes verifying that the device can
 109 join the network, send uplink messages, receive downlink messages, and maintain the required communication
 110 parameters.

111 2 Test Setup

112 The test environment and setup for interoperability testing of LoRaWAN Networks and LoRaWAN End-Devices
 113 involve a structured configuration that enables thorough evaluation of device capabilities and network compatibility. If
 114 supported, logs generated by any of the network elements are used to verify the expected behavior of the DUT.

115 The Interoperability test may be conducted in an environment similar to a Plug fest, where different network elements,
 116 i.e. End-Devices, Gateways, Network Servers, Join Servers and/or Application Servers are connected together. The
 117 test could also be conducted at LoRaWAN® vendor locations, member meetings, test labs, etc. as required by the
 118 End-Device, Network Server and Gateway vendors.

119 2.1 Test Environment Components



120 2.1.1 LoRaWAN Network Server

121 This component acts as the central point for processing messages from LoRaWAN End-Devices. It should be configured
 122 for the specific LoRaWAN version being tested (e.g., LoRaWAN 1.0.2, 1.0.4). This might involve using a public
 123 Network Server or setting up a private one, depending on test requirements.

124 Most Network Servers include the functionality of a Join Server and an Application Server. When planning your test
 125 protocol, this should be considered so as to minimize redundancies and wasted effort.

126 2.1.2 LoRaWAN End-Devices

127 LoRaWAN End-Devices could include sensors, actuators, or any other type of IoT device designed to communicate
128 over LoRaWAN. Each device may have different configurations and capabilities to test various features.

129 The End-Device must be an off-the-shelf device with the same configuration as the devices available on the market.

130 If the End-Device supports both Over-The-Air Activation (OTAA) and Activation by Personalization (ABP), either of
131 these 2 options can be followed for testing

- 132 1. Two separate End-Devices of the same configuration could be used for testing each of the modes OR
- 133 2. The complete Interoperability testing is performed using one activation mode. The End-Device is then
- 134 reconfigured to support the second activation mode, and the Device Activation tests are repeated.

135 2.1.3 LoRaWAN Gateway

136 A Gateway is required to bridge communication between End-Devices and the Network Server. It must be capable of
137 receiving and transmitting LoRa PHY packets and translate them to/from the protocol used by the gateway backhaul.

138 The Gateway must be an off-the-shelf device with the same configuration as the devices available on the market.

139 2.1.4 Testing Tools and Software

140 Tools for analyzing packets, network monitoring, and capturing logs are essential. This can include software that helps
141 with packet sniffing (e.g., Wireshark, TCPDump) and for over the air packet sniffing a combination of a common
142 SDR and software like Universal Radio Hacker, as well as any proprietary tools provided by the device manufacturer or
143 network solution provider.

144 2.2 Infrastructure Setup**145 2.2.1 Placement of Elements**

146 The Gateway must be placed in a position where it has adequate coverage for the End-Devices. Its location
147 may influence performance metrics such as signal strength and packet delivery rates. As this test plan is testing
148 interoperability, RF performance is not evaluated.

149 2.2.2 Device Configuration

150 Each End-Device must support either over-the-air (OTA) activation or activation by personalization (ABP) or both.

151 If the End-Device supports OTAA, it must be configured with the necessary parameters such as DevEUI, JoinEUI,
152 AppKey, (and NwkKey for LoRaWAN LW1.1.0+ devices). See Appendix for complete details.

153 If the End-Device supports ABP, it must be configured with parameters such as Network Session Key (NwkSKey),
154 Application Session Key (AppSKey), DevAddr, etc. See Appendix for complete details.

155 Various configurations and settings can be tested to evaluate how well they interact with the Network Server.

156 Note: Use of these parameters is also a part of the following test plan. You may want to wait for the provisioning
157 tests before programming these variables.

158 2.2.3 Frequency Plan

159 Tests must be conducted on the correct frequency plan permitted by regional regulations, which will vary depending
160 on the countries where the devices and networks will be deployed. The frequencies, bandwidth, and spreading factors
161 must be aligned with both the Network Server and the End-Device capabilities.

Channel Plan	Common Name	Channel Plan ID
EU863-870	EU868	1

Channel Plan	Common Name	Channel Plan ID
US902-928	US915	2
CN779-787	CN779	3
EU433	EU433	4
AU915-928	AU915	5
CN470-510	CN470	6
AS923-1	AS923	7
AS923-2	AS923-2	8
AS923-3	AS923-3	9
KR920-923	KR920	10
IN865-867	IN865	11
RU864-870	RU864	12
AS923-4	AS923-4	13

2.3 Implicitly Tested Test Cases

Implicitly Tested Test Cases refer to scenarios or conditions that are not explicitly defined or documented as part of the formal testing process but are nevertheless evaluated during testing activities. This may also include individual test steps within a test case. These test cases typically arise from the implicit understanding of the system's functionality or from the assumptions made by testers based on their knowledge of the application's expected behaviours. Some of the test cases contained in this test plan may in fact be implicitly tested by executing one or more of the other test cases in this document. It is the test engineer's prerogative to make this determination and act accordingly.

3 Interoperability Test Cases

3.1 Test Setup - Single End-Device - LNS Test Cases

The Interoperability Test Cases defined in the following sections can be used for testing any type of network element. The Network Servers may be live and actively handling commercial traffic, this is the preferred environment as long as the act of performing any of the tests in this document does not interfere with the normal operation of the network which could negatively impact paying customers.

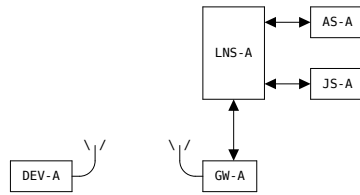
In these test cases, it is confirmed that

1. End-Devices can successfully connect to the LNS using ABP and OTAA methods.
2. Uplinks and downlinks are successfully exchanged between End-Device and LNS.
3. Switching of End-Device from one LNS to another.
4. LNS connects to two End-Devices at the same time.
5. Class B and Class C scenarios are tested.

	ED	GW	NS	JS	AS
Network Element	X	X	X	X	X

3.1.1 Components

- DEV-A: End-Device
- LNS-A: LoRaWAN Network Server
- GW-A: Gateway
- JS-A : Join Server
- AS-A : Application Server



187 3.1.2 Capabilities Required

- 188 ■ DEV-A must support:
 - 189 – CAP.1.1 (Trigger uplink): Ability to send an uplink message upon request.
 - 190 – CAP.1.2 (Trigger join): Ability to send a join request upon request.
 - 191 – CAP.1.3 (Trigger rejoin): Ability to send a re-join request upon request.
 - 192 – CAP.1.4 (Trigger restart): Ability to restart upon request.
 - 193 – CAP.1.5 (Trigger confirmed uplink): Ability to send a Confirmed uplink message upon request.
 - 194 – CAP.1.6 (Trigger unconfirmed uplink): Ability to send an Unconfirmed uplink message upon request.
- 195 ■ LNS-A must support:
 - 196 – CAP.2.3.1 (enable device): Ability to enable End-Device from the LNS upon request.
 - 197 – CAP.2.3.2 (disable device): Ability to disable End-Device from the LNS upon request.
 - 198 – CAP.2.4 (Trigger Network Server downlink): Ability to send a downlink message upon request.
- 199 ■ AS-A must support:
 - 200 – CAP.3.1 (Trigger Application Server downlink): Ability to send a downlink message upon request.

201 3.1.3 Events Expected

- 202 ■ DEV-A must generate:
 - 203 ■ EVT.1.1 (OTAA keys configured by device): Indicates that the OTAA End-Device has configured valid keys.
 - 204 ■ EVT.1.3 (DevAddr after JoinAccept in device)
 - 205 ■ EVT.1.4 (Uplink transmitted by device): Indicates that the End-Device has completed the transmission of a
 - 206 payload.
 - 207 ■ EVT.1.5 (Downlink received by device): Indicates that the End-Device has received the downlink payload.
- 208 ■ LNS-A must generate:
 - 209 ■ EVT.2.1.1 (Uplink received by NS): Indicates that the LNS has received the payload.
 - 210 ■ EVT.2.1.2 (Downlink sent by NS): Indicates that the LNS has sent the payload.
- 211 ■ AS-A must generate:
 - 212 ■ EVT.2.2.1 (Uplink received by AS): Indicates that the application server has received the payload.
 - 213 ■ EVT.2.2.2 (Downlink sent by AS): Indicates that the application server has sent the payload.
- 214 ■ JS-A must generate:
 - 215 ■ EVT.2.3.1 (Join request received by JS): Indicates that the join server has received the Join request.
 - 216 ■ EVT.2.3.2 (Join accept sent by JS): Indicates that the join server has sent the Join accept.

217 3.1.4 Prerequisites

- 218 ■ DEV-A is commissioned on LNS-A.
- 219 ■ DEV-A is programmed and configured with the required keys.
- 220 ■ DEV-A is reachable by GW-A.
- 221 ■ GW-A is operational and connected to the LNS-A.

- 222 ▪ LNS-A is configured with either an inbuilt JS-A and/or AS-A or LNS-A is connected to an independent JS-A
- 223 and/or AS-A.
- 224 ▪ JS-A is operational and ready to accept Join Requests from DEV-A via LNS-A.
- 225 ▪ AS-A is operational and ready to receive messages LNS-A.
- 226 ▪ Monitoring tools are available to capture logs.

227 3.2 Test Cases

228 3.2.1 Activation by Personalisation test

229 This section defines the Activation by Personalisation (ABP) test scenarios. These tests must be executed only if the
230 device is an ABP device. When an ABP device is restarted, it is confirmed that the End-Device sends an uplink on
231 restart, is assigned a valid DevAddr, is allocated a random and unique NwkSKey, is allocated a 128-bit AppKey per
232 application and preserves all its RF settings.

233 3.2.1.1 Test Steps

Scenario: Verify Reset parameters of the device for ABP

```
Given DEV-A supports Activation by Personalisation
And DEV-A is reset
When the operator triggers DEV-A to restart (CAP.1.4)
Then
  DEV-A must send an uplink (EVT.1.4)
And LNS-A must validate that DEV-A is assigned the DevAddr correctly where
  DevAddr = NetID + NwkAddr
And LNS-A must validate that DEV-A is allocated a random and unique NwkSKey
And LNS-A must validate that DEV-A is allocated a 128-bit AppKey per application
And (for LW1.0.4+ devices only) DEV-A must preserve all RF settings (RF channel,
  frame counters)
```

234 3.2.2 Over The Air Activation- Join-Rejoin Process

- 235 This section defines the Join and Rejoin test scenarios for all network elements. These tests are executed only when
236 the End-Device supports Over the Air Activation. The following scenarios are confirmed:
- 237 1. After the End-Device is reset, on triggered to send an uplink, the End-Device sends a Join-Request, the LNS
238 responds with a Join-Accept, the End-Device is assigned a valid DevAddr and the End-Device sends an uplink using
239 only the Default channels.
 - 240 2. When the End-Device is triggered to Re-join, the re-join must be successful. Apart from the checks mentioned
241 above, it is also confirmed that they End-Device communicates on the new channels with the CFList, sends the correct
242 MIC checksum for uplinks, properly uses the DLSettings and RxDelay defined in the Join-Accept.
 - 243 3. When RxDrOffset, Rx2Datarate, RxDelay is set in Join-Accept, the End-Device connects successfully and sends
244 uplinks to the LNS. It is also confirmed that the Duty cycle restrictions are met.

245 3.2.2.1 Test Steps

Scenario: Verify Join Procedure

```
Given the Prerequisites are met
And DEV-A is configured to use OTAA
And DEV-A is reset
When the operator triggers an uplink on DEV-A (CAP.1.1)
Then
```

DEV-A must send a Join Request to the LNS-A (EVT.1.4)
And LNS-A must respond with Join Accept to DEV-A with NetID, DevAddr, JoinNonce, DLSettings, RXDelay and if possible, no CFList (EVT.2.3.2)
And DEV-A must be assigned a valid DevAddr by the LNS-A
And DEV-A must send an uplink (EVT.1.4)
And if no CFList was sent in the Join Accept, only the default channels must be used to send the uplink

Scenario: Verify Re-join Process

Given DEV-A has joined the LNS-A network

When the operator initiates the OTAA Join Rejoin procedure on DEV-A for LW1.1+ (CAP.1.3) or triggers another Join request for LW1.0.2 and LW1.0.4(CAP.1.2)

Then

DEV-A must send a Join Request (LW1.0.2/1.0.4) or Rejoin Request (LW1.1+) to LNS-A with the stored credentials (EVT.1.4)

And LNS-A must send a Join-Accept must respond with Join Accept to DEV-A with Net-ID, DevAddr, AppNonce, DLSettings, CFList (if applicable), RXDelay (EVT.2.3.2)

And DEV-A must be assigned a valid DevAddr by the LNS-A

And DEV-A must send an uplink using the new channels with the CFList (if applicable)

And DEV-A must send the correct MIC checksum for uplinks

Given LNS-A can send Confirmed downlinks on request

When the operator triggers DEV-A to send an uplink (CAP.1.1)

Then

LNS-A must send a Confirmed downlink to DEV-A

And DEV-A must send an acknowledgement to LNS-A.

If the ACK is not automatically sent,

the operator triggers DEV-A to send an uplink (CAP.1.1)

And DEV-A must properly use the DLSettings in the uplink as defined in the Join-Accept

And DEV-A must properly use the RxDelay in the uplink as defined in the Join-Accept

Scenario: Verify different RxDrOffset, Rx2Datarate, RxDelay

Given DEV-A has joined the LNS-A network

When the operator initiates the OTAA Join Rejoin procedure on DEV-A for LW1.1+ (CAP.1.3) or triggers another Join request for LW1.0.2 and LW1.0.4(CAP.1.2)

Then

DEV-A must send a Join Request (LW1.0.2/1.0.4) or Rejoin Request (LW1.1+) to LNS-A with the stored credentials (EVT.1.4)

And LNS-A must send a Join-Accept must respond with Join Accept to DEV-A with a different RxDrOffset, Rx2Datarate, RxDelay (EVT.2.3.2)

And DEV-A must be assigned a valid DevAddr by the LNS-A

And DEV-A must send an uplink (EVT.1.4)

246 3.2.3 Uplink Transmission

247 This section defines the Uplink Transmission test scenarios. It is confirmed that the End-Device sends Unconfirmed
248 and Confirmed uplinks and the LNS responds with acknowledgements for Confirmed uplinks.

249 3.2.3.1 Test Steps

Scenario: Verify Unconfirmed Uplink transmission

Given DEV-A is ready to send an application specific uplink payload to LNS-A/AS-A
And DEV-A can send an Unconfirmed uplink payload to LNS-A
When the operator triggers DEV-A to send an application specific Unconfirmed uplink payload to LNS-A/AS-A (CAP.1.6)
Then
DEV-A must send an uplink that has the correct message type = Unconfirmed to LNS-A/AS-A (EVT.1.4)
And LNS-A/AS-A must receive the uplink (EVT.2.1.1)

Scenario: Verify Confirmed Uplink transmission

Given DEV-A is ready to send an application specific uplink payload to LNS-A/AS-A
And DEV-A can send a Confirmed uplink payload to LNS-A
When the operator triggers DEV-A to send an application specific Confirmed uplink payload to LNS-A/AS-A (CAP.1.5)
Then
DEV-A must send an uplink that has the correct message type = Confirmed to LNS-A/AS-A (EVT.1.4)
And LNS-A must send an acknowledgement with incremented FCntDown to DEV-A (EVT.2.1.2)
And DEV-A must receive the acknowledgement and send the next uplink with incremented FCntUp to LNS-A (EVT.1.4)

250 3.2.4 Downlink Transmission

251 This section defines the Downlink Transmission test scenarios. It is confirmed that the End-Device successfully receives
252 Unconfirmed and Confirmed downlinks from the LNS. For Confirmed downlinks, it is verified that the End-Device
253 sends acknowledgements.

254 3.2.4.1 Test Steps**Scenario: Verify Unconfirmed Downlink transmission**

Given DEV-A is ready to send an uplink payload to LNS-A
And LNS-A can send Unconfirmed downlinks to DEV-A upon request
When the operator triggers DEV-A to send an uplink payload to LNS-A (CAP.1.1)
Then
DEV-A must send an uplink to LNS-A (EVT.1.4)

When the operator triggers LNS-A to send an UNCONFIRMED downlink (EVT.2.1.2)
Then DEV-A must receive the downlink (EVT.1.5)

Scenario: Verify Confirmed Downlink transmission

Given DEV-A is ready to send an uplink payload to LNS-A
And LNS-A can send CONFIRMED downlinks to DEV-A upon request
When the operator triggers DEV-A to send an uplink payload to LNS-A (CAP.1.1)
Then
DEV-A must send an uplink to LNS-A (EVT.1.4)

When the operator triggers LNS-A to send a CONFIRMED downlink with a correct FCntDown (EVT.2.1.2)

Then DEV-A must send an acknowledgement to LNS-A (EVT.1.4)

255 3.2.5 Class B tests

256 This section defines the Class B test scenarios. It is confirmed that

257 1. When the End-Device switches from Class A to Class B, PingSlotInfoReq is sent on receiving a beacon. LNS
258 responds with the PingSlotInfoAns.

259 2. In Class B mode, the End-Device sets the Class B bit of FCtrl field to 1 for uplinks.

260 3.2.5.1 Test Steps

Scenario: Switch between Class A and B and vice versa

Given LNS-A is sending continuous beacons

And DEV-A supports Class B

And DEV-A is currently in Class A mode

When the operator triggers DEV-A to switch from Class A to Class B

Then

DEV-A must send a PingSlotInfoReq on receiving a beacon (EVT.1.4)

And LNS-A sends a PingSlotInfoAns to DEV-A (EVT.2.1.2)

When the operator triggers DEV-A to send an uplink payload to LNS-A (CAP.1.1)

Then

DEV-A must send an uplink with Class B bit of FCtrl field set to 1 (EVT.1.4)

261 3.2.6 Class C tests

262 This section defines the Class C test scenarios. It is confirmed that in Class C mode, when the LNS sends a downlink
263 randomly after uplink, the End-Device receives it successfully.

264 3.2.6.1 Test Steps

Scenario: Verify CLASS C tests

Given DEV-A supports Class C

And DEV-A is a LW1.0.4+ device

And DEV-A supports OTAA

And LNS-A has provisioned DEV-A as a Class C device

When the operator triggers a Join-Request on DEV-A (CAP.1.2)

Then

DEV-A must send a Join Request to the LNS-A (EVT.1.4)

And LNS-A must respond with Join Accept to DEV-A (EVT.2.3.2)

And DEV-A must send a Confirmed uplink (EVT.1.4)

And LNS-A must respond with an acknowledgement (EVT.2.1.2)

Given DEV-A supports Class C

And DEV-A is a LW1.0.4+ device

And DEV-A support supports Activation by Personalisation

And LNS-A has provisioned DEV-A as a Class C device

And DEV-A is reset

When the operator triggers DEV-A to restart (CAP.1.4)

Then

DEV-A must send a Confirmed uplink (EVT.1.4)

And LNS-A must respond with an acknowledgement (EVT.2.1.2)

Given the tests mentioned above succeed

And LNS-A can send a downlink randomly after uplink

When the operator triggers the LNS-A to send a downlink randomly after uplink (CAP.2.4)

Then

LNS-A must send a downlink randomly after uplink (EVT.2.1.2)

And DEV-A must receive the downlink (EVT.1.5)

265 4 Appendix

266 4.1 Capability Tree

267 When running a test, the operator is frequently asked to perform an operation. It is known in advance if the DUT is
268 able to perform this operation, otherwise the test is impossible. We are therefore defining as *Capability* a feature on
269 the DUT that the operator can use in order to perform the test.

270 In order to efficiently and unambiguously encode the capabilities each DUT we are using a pre-defined list of capabilities,
271 described in this annex. Each capability has a unique identifier in the form of "CAP.x.x..." notation, for example
272 CAP.2.1.

273 The capabilities are layed out in a hierarchical tree. The presence of a higher-order capability automatically implies the
274 presence of child capabilities in the DUT.

275 This means that if the DUT implements the root capability, it is the golden standard.

276 4.1.1 CAP.1 : End device capabilities

- 277 ▪ CAP.1.1 (trigger uplink): Can be triggered to send an uplink at operator's request, using any means possible
278 (eg. power-cycle, trigger a sensor event, etc.)
- 279 ▪ CAP.1.2 (trigger join): Can be triggered to send a join request at operator's request, using any means possible
280 (eg. power-cycle, trigger a sensor event, etc.)
- 281 ▪ CAP.1.3 (trigger rejoin): Can be triggered to send a rejoin at operator's request, using any means possible (eg.
282 LW1.1.x+ Rejoin-Request, power-cycle, trigger a sensor event, etc.)
- 283 ▪ CAP.1.4 (trigger restart): Can be triggered to restart the device
- 284 ▪ CAP.1.5 (trigger confirmed uplink): Can be triggered to send a Confirmed uplink at operator's request, using
285 any means possible (eg. power-cycle, trigger a sensor event, etc.)
- 286 ▪ CAP.1.6 (trigger unconfirmed uplink): Can be triggered to send an Unconfirmed uplink at operator's request,
287 using any means possible (eg. power-cycle, trigger a sensor event, etc.)

288 4.1.2 CAP.2 : Device-Level LNS Capabilities

- 289 ▪ CAP.2.1 (retrieve device identity): All properties of the device identity can be retrieved by the LNS.
 - 290 – CAP.2.1.1 (retrieve device identity from NS): Session details relevant to the NS for a given device can be
291 seen/exported.
 - 292 * CAP.2.1.1.1 (retrieve device NwkSKey from NS): Network Session Key (NwkSKey) of a specific
293 device can be seen/exported .
 - 294 * CAP.2.1.1.2 (retrieve device DevAddr from NS): Current device address (DevAddr) of a specific
295 device can be seen/exported.

- 296 * CAP.2.1.1.3 (retrieve device FcntUp from NS/FcntDn): Current device frame counters (Fc-
- 297 ntUp/FcntDn) can be seen/exported.
- 298 * CAP.2.1.1.4 (retrieve device MAC config from NS): All of MAC configuration (regional settings, RX
- 299 windows, etc.) can be seen/exported.
- 300 – CAP.2.1.2 (retrieve root JoinKey from JS): The root Application Key can be retrieved from the Join
- 301 Server.
- 302 – CAP.2.1.3 (retrieve AppSKey from AS): The Application Session Key can be retrieved from the Application
- 303 Server.
- 304 ■ CAP.2.2 (update device identity) : All properties of the device identity can be updated in the LNS. If a device
- 305 does not exist, it assumes that the device can be created.
- 306 – CAP.2.2.1 (update device identity to the NS): Session details relevant to the NS for a given device can
- 307 be created/updated.
- 308 * CAP.2.2.1.1 (update device NwkSKey to the NS): Network Session Key (NwkSKey) of a specific
- 309 device can be created/updated .
- 310 * CAP.2.2.1.2 (update device DevAddr to the NS): Current device address (DevAddr) of a specific
- 311 device can be created/updated.
- 312 * CAP.2.2.1.3 (update device FcntUp/FcntDn to the NS): Current device frame counters (Fc-
- 313 ntUp/FcntDn) can be created/updated.
- 314 * CAP.2.2.1.4 (update device MAC config to the NS): All of MAC configuration (regional settings,
- 315 RX windows, etc.) can be created/updated.
- 316 – CAP.2.2.2 (update root JoinKey to the JS): The root Application Key can be created/updated to the
- 317 Join Server.
- 318 – CAP.2.2.3 (update AppSKey to the AS): The Application Session Key can be retrieved from the Application
- 319 Server.
- 320 ■ CAP.2.3 (enable/disable device): The operator should be able to enable or disable a device on the LNS.
- 321 – CAP.2.3.1 (enable device)
- 322 – CAP.2.3.2 (disable device)
- 323 ■ CAP.2.4 (trigger Network Server downlink): Can be triggered to send an downlink at operator's request

324 4.1.3 CAP.3 : Application Server capabilities

- 325 ■ CAP.3.1 (trigger Application Server downlink): Can be triggered to send an downlink at operator's request,
- 326 using the Application Server Management Interface
- 327 ■ CAP.3.2 (trigger Device Firmware Upgrade): Can be triggered to send downlink messages at operator's request,
- 328 using the Application Server Management Interface, to upgrade the firmware version on the device.

329 4.2 Events Tree

330 When running a test, the operator must identify if the expected outcome has been achieved. Since this outcome may
 331 vary across implementations, we are definition as an *Event*, a well-defined outcome that the DUT is producing.

332 In an ideal scenario, the DUT is implementing an API that emit such events, that could be observed by the operator,
 333 or the test automation tool.

334 However, since such API might not be available, the events are crafted in such a way so that the operator can observe
 335 manually.

336 Similar to the capabilities, event identifiers are following the pattern of "EVT.x.x.x..." notation. For example
 337 EVT.1.1. Each event is also accompanied with the respective Google Protobuf schema, that can be used to create
 338 cross-language interfaces to that event.

339 4.2.1 EVT.1 : End device event

- 340 ■ EVT.1.1 (OTAA keys configured by device): Indicates that the OTAA end device has configured valid keys.
- 341 This may be identified by checking the device logs or an indication on the end device management interface.

```

message EVT1_1_OTADeviceKeysConfigured {
    // LoRaWAN version supported
    LWVersion version = 1

    //Device keys applicable for all LW versions
    bytes dev_eui = 2
    bytes app_key = 3

    //Device keys applicable only for LW1.0.2 devices
    optional bytes app_eui = 4

    //Device keys applicable only for LW1.0.4+ devices
    optional bytes join_eui = 5
}

```

- 342 ▪ EVT.1.2 (ABP keys configured by device): Indicates that the ABP end device has configured valid keys. This
- 343 may be identified by checking the device logs or an indication on the end device management interface.

```

message EVT1_2_ABPDeviceKeysConfigured {
    // LoRaWAN version supported
    LWVersion version = 1

    //Device keys applicable for all LW versions
    bytes dev_addr = 1
    bytes nwk_skey = 2
    bytes app_skey = 3

    //The following fields are applicable only for LW1.0.2 devices
    int32 f_cnt = 4
    bytes channels = 5
}

```

- 344 ▪ EVT.1.3 (DevAddr after JoinAccept in device): Indicates that the OTAA end device has a valid DevAddr after
- 345 joining the network. This may be identified by checking the device logs or an indication on the end device
- 346 management interface.

```

message EVT1_3_DevAddrAfterJoin {
    //The DevAddr configured
    bytes dev_addr = 1
}

```

- 347 – EVT.1.4 (Uplink transmitted by device): Indicates that the end device has completed the transmission of
- 348 a payload. This may be a visual indication on the device, an indication on the display, or the respective
- 349 payload sent over the UART line.

```

message EVT1_4_DataSent {
    Timestamp ts = 1
    // The payload the device sent
    bytes payload = 2
    // The LoRaWAN MAC frame constructed by the device
    MACFrame frame = 3
}

```

- 350 ▪ EVT.1.5 (Downlink received by device): Indicates that the device has received the downlink payload from the
- 351 server. This may trigger state changes on the device, actions taken (like updating a variable) or responses
- 352 generated.

```
message EVT1_5_DataReceived {
    Timestamp ts = 1
    // The payload the device received
    bytes payload = 2
}
```

353 4.2.2 EVT.2 : LNS Events

354 4.2.2.1 EVT.2.1. : Network Server Events

- 355 ■ EVT.2.1.1 (Uplink received by NS): Indicates that the network server has received the payload.

```
message EVT2_1_1_NSMACFrameRx {
    Timestamp ts = 1
    // The LoRaWAN Mac frame received by the network server
    MACFrame frame = 2
}
```

- 356 ■ EVT.2.1.2 (Downlink sent by NS): Indicates that the network server has sent the payload.

```
message EVT2_1_2_NSMACFrameTx {
    Timestamp ts = 1
    // The LoRaWAN MAC frame sent by the network server
    MACFrame frame = 2
}
```

357 4.2.2.2 EVT.2.2. : Application Server Events

- 358 ■ EVT.2.2.1 (Uplink received by AS): Indicates that the application server has received the unencrypted device
- 359 payload and performed any associated actions, for example: triggering alerts, storing sensor data.

```
message EVT2_2_1_ASUplinkRx {
    Timestamp ts = 1
    // The payload the device sent
    bytes payload = 2;
}
```

- 360 ■ EVT.2.2.2 (Downlink sent by AS): Indicates that the application server has sent the payload to the network
- 361 server.

```
message EVT2_2_2_ASDownlinkTx {
    Timestamp ts = 1
    // The payload the device sent
    bytes payload = 2;
}
```

- 362 ■ EVT.2.3.1 (Join request received by JS): Indicates that the join server has received the join request.

363

```
message EVT2_3_1_JSJoinRequestRx {
    Timestamp ts = 1
    // The payload the JS received
    bytes payload = 2;
}
```

364

- 365 – EVT.2.3.2 (Join accept sent by JS): Indicates that the join server has sent the join accept.

366

```
message EVT2_3_2_JSJoinAcceptTx {  
    Timestamp ts = 1  
    // The payload the JS sent  
    bytes payload = 2;  
}
```

367

368 4.3 Shared Event Fields

369 To avoid duplicating information, the events on Events Tree are referring to messages that are re-used across multiple
370 events.

371 This section contains their definition:

372

```
message MACFrame {  
    // The LoRaWAN uplink is required in order to check the  
    // contents of the frame counters, or the MAC commands.  
    oneof {  
        MAC_Payload mhdr = 1  
        Join_Request mhdr = 2  
        Join_Accept mhdr = 3  
    }  
    // The encrypted payload of the message  
    bytes encPayload = 2  
}
```

```
message MACPayload {  
    bool confirmed = 1;  
    bytes dev_addr = 2;  
    int32 f_ctrl = 3;  
    int32 f_cnt = 4;  
    bytes f_opts = 5;  
}
```

```
message Join_Request {  
    // To distinguish between confirmed/unconfirmed  
    int32 m_type = 1;  
}
```

```
message Join_Accept {  
    // To distinguish between confirmed/unconfirmed  
    int32 mType = 1;  
}
```

```
enum LWVersion {  
    // To enumerate the LoRaWAN version supported  
    LW_1_0_2 = 1;  
    LW_Post_1_0_4 = 2;  
};
```

373

374 4.4 Full session context elements

375 The full session context elements depend on the LoRaWAN specification version.

376

377

LW Ver- sion	Activation method	Parameters
1.0.2	OTA	DevAddr, AppEUI, AppKey
1.0.2	ABP	DevAddr, NwkSKey, AppSKey Upon first boot and following a reset, all personalized end-devices SHALL have all available channels for that region enabled. ABP end-devices SHALL store the frame counters persistently (e.g., in non-volatile memory)
1.0.4	OTA	DevAddr, JoinEUI, AppKey
1.0.4	ABP	DevAddr, NwkSKey, AppSKey Upon first boot and following a reset, all personalized end-devices SHALL have all available channels for that region enabled. ABP end-devices SHALL store the frame counters persistently (e.g., in non-volatile memory)
1.2.0	OTA	DevAddr, JoinEUI, AppKey
1.2.0	ABP	DevAddr, NwkSKey, AppSKey

378